



UACM

Universidad Autónoma
de la Ciudad de México

NADA HUMANO ME ES AJENO



JUEGO_DE_CARTAS (Construcción y evolución de un juego de cartas en Java)

Licenciatura en Ingeniería de Software

Materia: Construcción y Evolución de Software

Integrantes:

- Aguilar Dominguez Leslie Joseline
- García Quintero Ángel Antonio
- Perez Copado Carmen Andrea
- Ramírez Gallardo Ivan Enrique

HISTORIAL DE VERSIONES

Fecha	Versión	Descripción	Autores
24/02/26	1.0	Se entrega el avance de documentación y avance de código	• Aguilar Dominguez Leslie Joseline • García Quintero Ángel Antonio • Perez Copado Carmen Andrea • Ramírez Gallardo Ivan Enrique
17/03/26	2.0	Se agregan nuevos requisitos funcionales (CARTAS ESPECIALES)	• Aguilar Dominguez Leslie Joseline • García Quintero Ángel Antonio • Perez Copado Carmen Andrea • Ramírez Gallardo Ivan Enrique
14/04/26	3.0	Refactorización del sistema: implementación de múltiples jugadores, turnos dinámicos, clase Player e inteligencia artificial básica	• Aguilar Dominguez Leslie Joseline • García Quintero Ángel Antonio • Perez Copado Carmen Andrea • Ramírez Gallardo Ivan Enrique
30/04/26	4.0	Documentación técnica integral (Javadoc) y optimización de arquitectura.	• Aguilar Dominguez Leslie Joseline • García Quintero Ángel Antonio • Perez Copado Carmen Andrea • Ramírez Gallardo Ivan Enrique

Índice

Introducción	4
Misión	4
Visión.....	5
Alcance	5
¿Qué se mejoro y porque?	7
Roles	7
Requisitos funcionales	7
Versión 1.....	7
Versión 2.....	8
Versión 3.....	8
Requisitos no funcionales	9
Diagramas	10
DIAGRAMA DE CASOS DE USO	10
DIAGRAMAS DE SECUENCIA.....	11
DIAGRAMA DE ESTADOS	14
Diagrama UML.....	15
RELACIONES UML.....	16
Código.....	17

Main	¡Error! Marcador no definido.
Game	¡Error! Marcador no definido.
Hand.....	¡Error! Marcador no definido.
Card.....	¡Error! Marcador no definido.
Deck	¡Error! Marcador no definido.
Entrega	33

Introducción

Desarrollar una aplicación en Java que implemente una versión básica del juego UNO en consola, permitiendo a un jugador humano competir contra la computadora utilizando únicamente cartas numéricas.

El sistema servirá como base evolutiva para versiones posteriores del proyecto.

Misión

Desarrollar un juego de cartas llamado JUEGO_DE_CARTAS en Java, con interfaz de consola, que permita una partida entre un jugador humano y la computadora.

Versión 1:

- Baraja con 4 colores y números del 0 al 9
- Dos cartas por número por color
- Repartir 7 cartas a cada participante
- Carta inicial en mesa
- Turnos alternados
- Validar jugadas por color o número
- Robar cuando no haya jugadas
- Determinar ganador

Versión 2 (adicional):

- Incorporar cartas especiales
- Implementar efectos de cartas
- Nuevas reglas de juego
- Penalizaciones
- Manejo de efectos

Versión 3 (adicional):

- Soporte para múltiples jugadores (2 a 4)
- Implementación de inteligencia artificial básica
- Sistema de turnos dinámicos
- Refactorización del código
- Mejora en la estructura del sistema

Versión 4 (adicional):

- 1. Estandarización Profesional: Implementación integral del estándar Javadoc en todo el código fuente para eliminar deuda técnica y advertencias de compilación.
- Mantenibilidad y Limpieza: Refactorización de constructores y firmas de métodos para asegurar que el sistema sea comprensible y fácil de mantener por otros desarrolladores.
- Consolidación de Pruebas Unitarias: Documentación y categorización de la suite de pruebas en PruebasUno.java para garantizar la estabilidad de las reglas del juego.

Visión

Convertirse en un proyecto de referencia dentro de la asignatura que demuestre buenas prácticas de ingeniería de software: modularidad, mantenibilidad y facilidad de evolución. A futuro, se espera que JUEGO_DE_CARTAS pueda extenderse con nuevas funcionalidades como cartas especiales (reversa, bloqueo, toma dos, comodines), soporte para más jugadores e incluso una interfaz gráfica, sin necesidad de reestructurar su núcleo. De esta forma, el proyecto servirá como base didáctica para enseñar conceptos de diseño, evolución y mantenimiento de software, mostrando cómo una arquitectura sólida permite adaptarse a nuevos requerimientos de manera eficiente y ordenada.

Alcance

Versión 1:

- Juego en consola
- Cartas numéricas (0-9)
- Cuatro colores
- Reglas básicas
- Un jugador vs computadora

Versión 2 (adicional):

- Cartas especiales
- Baraja completa de 108 cartas
- Efectos de cartas
- Nuevas reglas
- Penalizaciones

Versión 3 (adicional):

- Juego multi-jugador (2-4 jugadores)
- Jugadores humanos y bots
- Turnos dinámicos con dirección
- Reglas completas del juego UNO
- Interacción mejorada en consola

Versión 4 (adicional):

- Documentación de API: Generación de un sitio web técnico (vía Javadoc) que describe cada clase, atributo y método del sistema.
- Calidad de Código (Zero Warnings): Alcance de un estado de "Cero Advertencias" en la generación de documentación, corrigiendo 34 omisiones de comentarios técnicos.
- Verificación y Validación: Cobertura de pruebas para casos de "Caja Negra" (reglas), "Decisión" (lógica del motor) y "Excepciones" (errores de ejecución).
- Arquitectura de Reglas: Consolidación del RuleEngine como un módulo independiente, permitiendo que la lógica de las cartas especiales sea reutilizable y clara.

¿Qué se mejoró y por qué?

Mejoras Realizadas (v3.0 a v4.0)

- Escalabilidad de Jugadores: Se pasó de un sistema de dos jugadores a un sistema multi-jugador (2 a 4 jugadores) con estructuras dinámicas (ArrayList<Player>).
- Control de Flujo: Implementación de turnos mediante índice y dirección para soporte de cartas de Reversa. Documentación Técnica (Javadoc): Se implementó el estándar Javadoc en todas las clases, eliminando las 34 advertencias de compilación para una entrega profesional.
- Estandarización de Pruebas: El archivo PruebasUno.java fue documentado y categorizado para validar reglas de Caja Negra, decisiones lógicas y excepciones.
- Refactorización de Mantenibilidad: Se añadieron constructores explícitos y comentados en clases como RuleEngine y Main para cumplir con las normas de calidad de Java.

¿Por qué se realizaron estos cambios?

- Reducción de Deuda Técnica: Eliminar las advertencias de documentación asegura un producto de software terminado y de alta calidad.
- Mantenibilidad: Se busca que cualquier desarrollador pueda comprender la lógica del motor de reglas sin necesidad de leer línea por línea el código fuente.
- Escalabilidad: Las estructuras dinámicas reducen el acoplamiento y facilitan futuras mejoras, como una interfaz gráfica.
- Robustez: La estandarización de pruebas asegura que futuras versiones no rompan la lógica funcional establecida.

Roles

Rol	Integrantes
Programador principal	García Quintero Ángel Antonio
QA/Testing	Ramírez Gallardo Ivan Enrique
Documentadora	Aguilar Dominguez Leslie Joseline
Integradora	Perez Copado Carmen Andrea
Desarrolladores	Todos

Requisitos funcionales

Versión 1

ID	Requisito
RF1	Crear baraja con 4 colores, números 0-9, dos cartas por número
RF2	Repartir 7 cartas a jugador y 7 a computadora, poner carta inicial
RF3	Mostrar en consola: carta en mesa, cartas del jugador, mensajes
RF4	Turno del jugador: puede elegir carta por índice o robar
RF5	Turno de computadora: juega primera carta válida o roba
RF6	Validar jugadas: debe coincidir color o número
RF7	Terminar cuando alguien se quede sin cartas

Versión 2

ID	Requisito
RF1	El sistema deberá incorporar cartas especiales: Reversa, Salto, Roba 2, Roba 4, Comodín
RF2	Una carta podrá jugarse si: coincide color, coincide número, coincide tipo, o es comodín/roba4

RF3	SALTO: El siguiente jugador pierde turno
RF4	REVERSA: En modo 2 jugadores se comporta como SALTO
RF5	ROBA2: El oponente roba 2 cartas
RF6	ROBA4: El oponente roba 4 cartas y pierde turno
RF7	COMODÍN: El jugador elige el nuevo color activo
RF8	Cuando un jugador tenga una sola carta, el sistema deberá mostrar mensaje: "¡Debes decir UNO!"
RF9	Si no confirma → roba 2 cartas
RF10	Si un jugador roba una carta: solo roba UNA carta; si es válida puede jugarla; si no es válida → pierde turno

Versión 3

ID	Requisito
RF1	Permitir seleccionar entre 2 y 4 jugadores
RF2	Gestionar múltiples jugadores mediante una estructura dinámica
RF3	Implementar turnos dinámicos con dirección del juego
RF4	Adaptar reglas de cartas a múltiples jugadores
RF5	Implementar inteligencia artificial básica
RF6	Mostrar el estado global del juego
RF7	Detectar condición de victoria en múltiples jugadores
RF8	Implementar correctamente la lógica de robar carta

Versión 4

ID	Requisito
RF1	Permitir seleccionar entre 2 y 4 jugadores
RF2	Gestionar múltiples jugadores mediante una estructura dinámica
RF3	Implementar turnos dinámicos con dirección del juego
RF4	Adaptar reglas de cartas especiales (Salto, Reversa, Roba 2, Roba 4, Comodín) a múltiples jugadores.
RF5	Implementar inteligencia artificial básica para oponentes.
RF6	Detectar condición de victoria y anunciar al ganador.

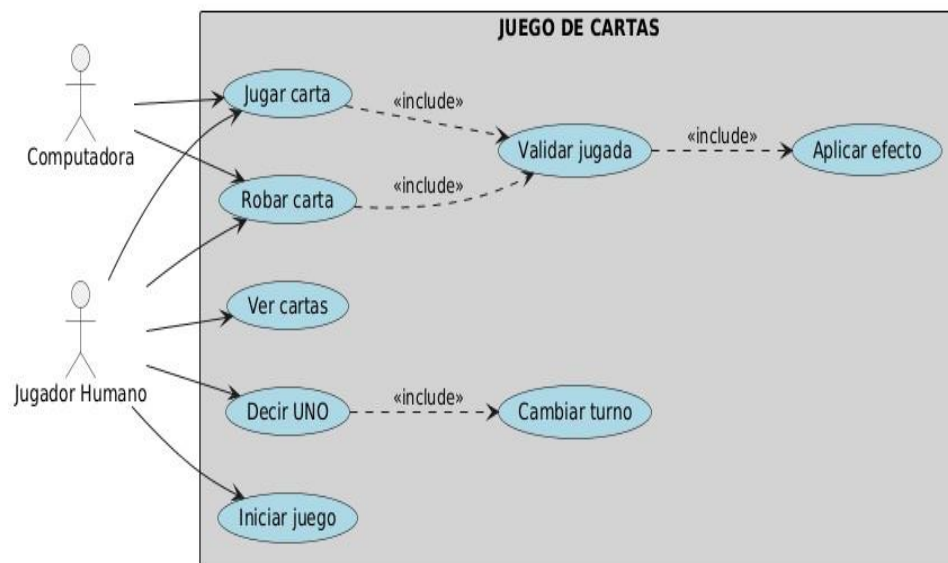
Requisitos no funcionales

ID	Requisito
RNF1	Programar en Java
RNF2	Interfaz solo por consola
RNF3	Usar programación orientada a objetos
RNF4	Código comentado y legible
RNF5	Control de versiones con Git
RNF6	No se permitirá lógica duplicada para efectos
RNF7	El código debe estar comentado y organizado
RNF8	Compilación limpia con 0 advertencias de documentación. QA/TESTING (v4.0) Se utiliza PruebasUno.java para las siguientes validaciones: Caja Negra: Validación de efectos de cartas especiales (Roba 4, Reversa, Comodín). Decisiones: Verificación de la lógica de RuleEngine para jugadas válidas por color, número o tipo. Excepciones: Validación de robustez ante accesos a índices inválidos en la mano del jugador.

Diagramas

DIAGRAMA DE CASOS DE USO

Diagrama de Casos de Uso - JUEGO DE CARTAS v2



DIAGRAMAS DE SECUENCIA

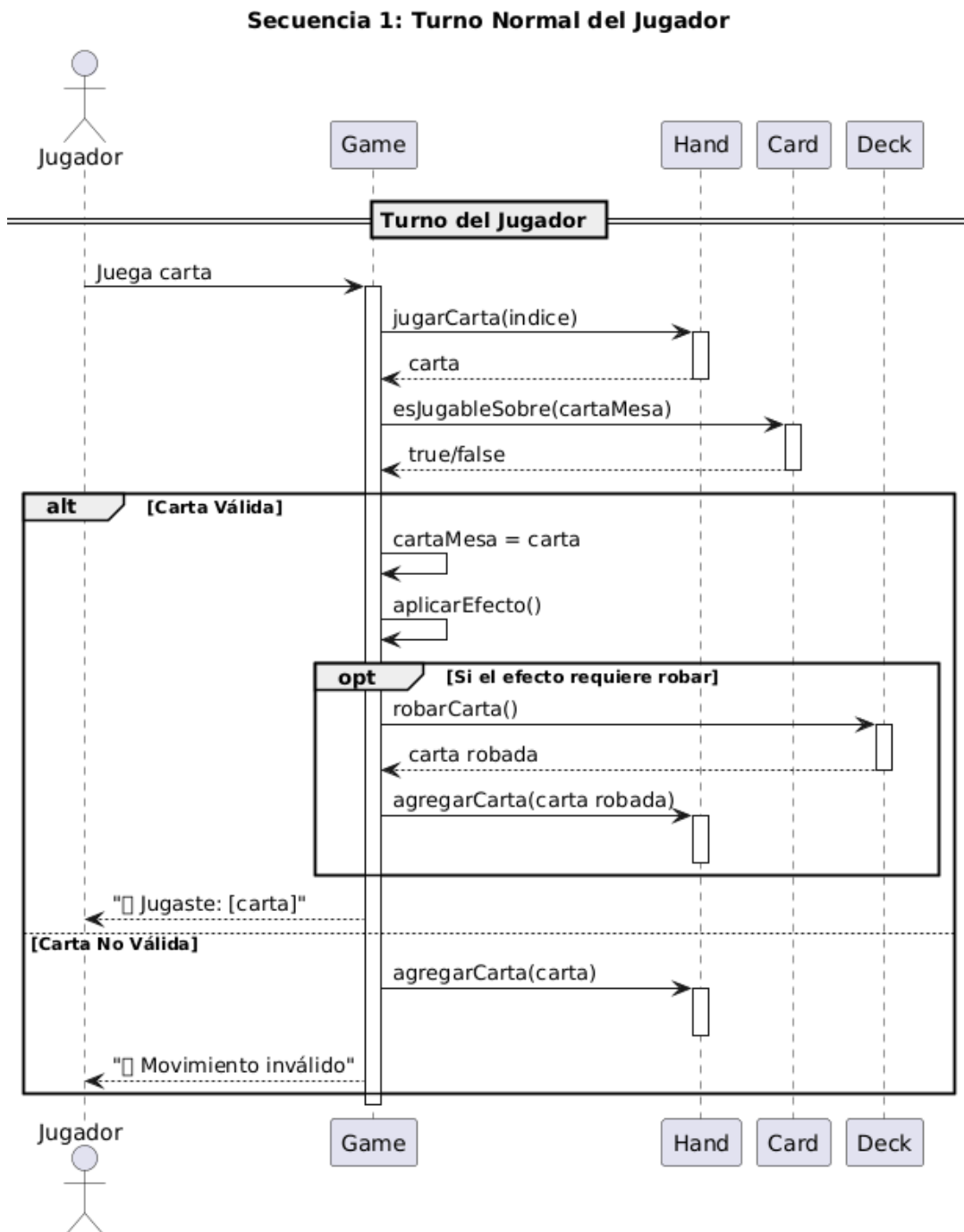
Secuencia 1: Turno Jugador

Jugador → Game: Juega carta

Game → Hand: Validar carta

Game → Card: Aplicar efecto

Game → Jugador: Resultado



Secuencia 2: Jugador roba carta (Caso UNO)

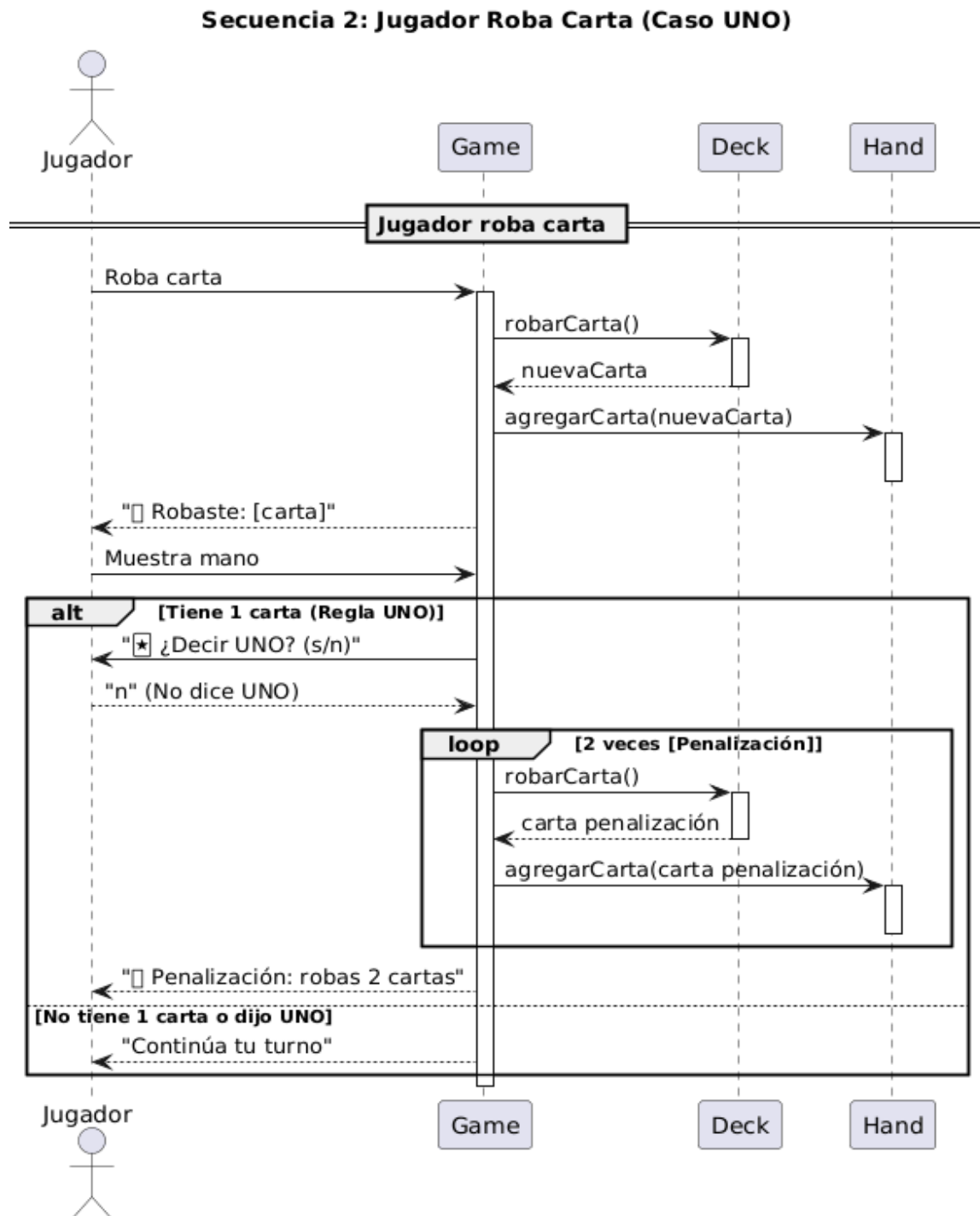
Jugador → Game: Roba carta

Game → Deck: Robar carta

Game → Jugador: Entrega carta

Jugador → Game: ¿Decir UNO?

Game → Jugador: Penalización si no dijo UNO



Secuencia 3: Aplicar efecto ROBA2

Game → Deck: Robar carta x2

Deck → Game: Carta 1, Carta 2

Game → Hand oponente: Agregar cartas

Secuencia 3: Aplicar Efecto ROBA2

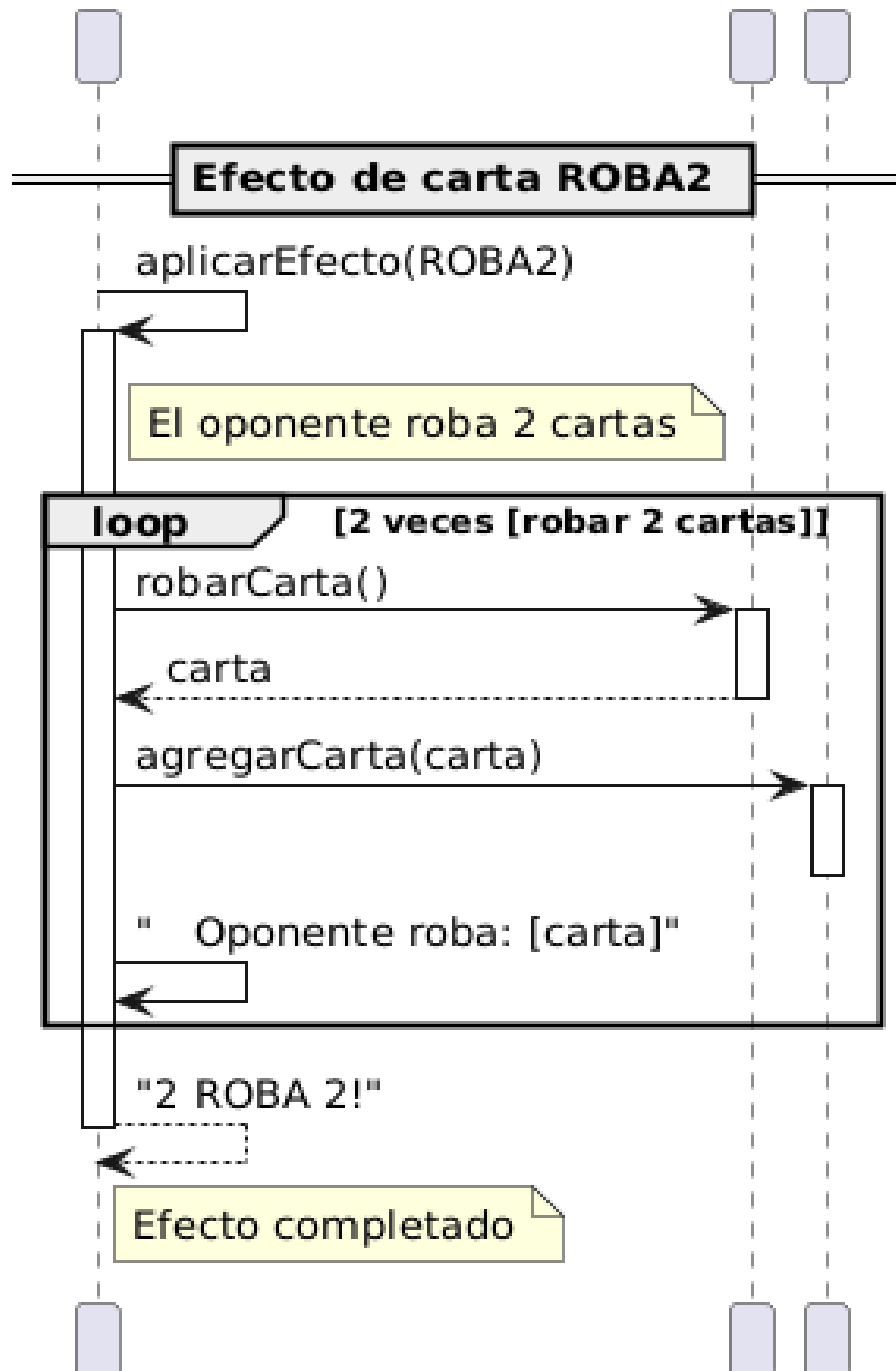


DIAGRAMA DE ESTADOS

DIAGRAMA DE ESTADOS - JUEGO DE CARTAS v2

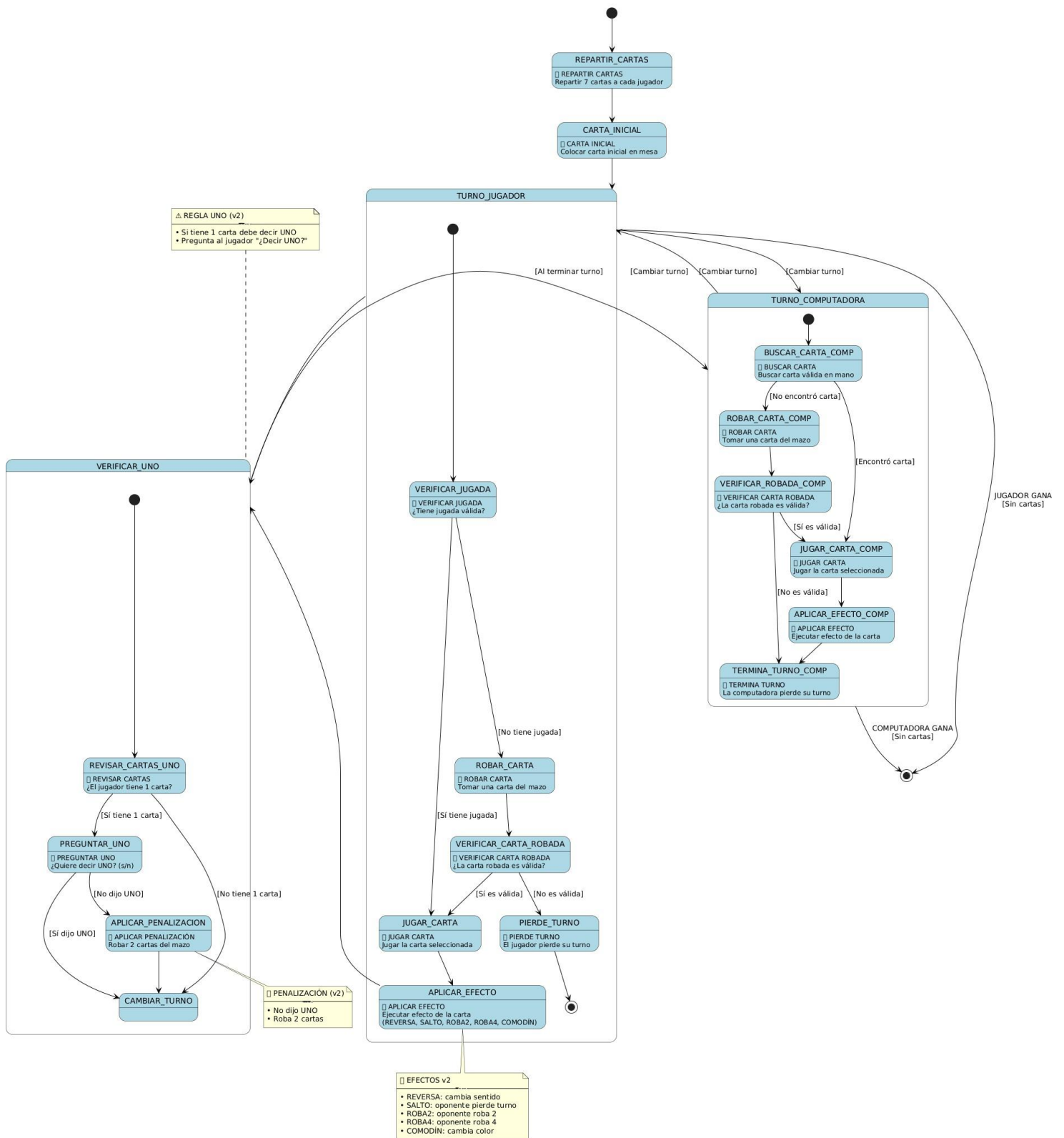
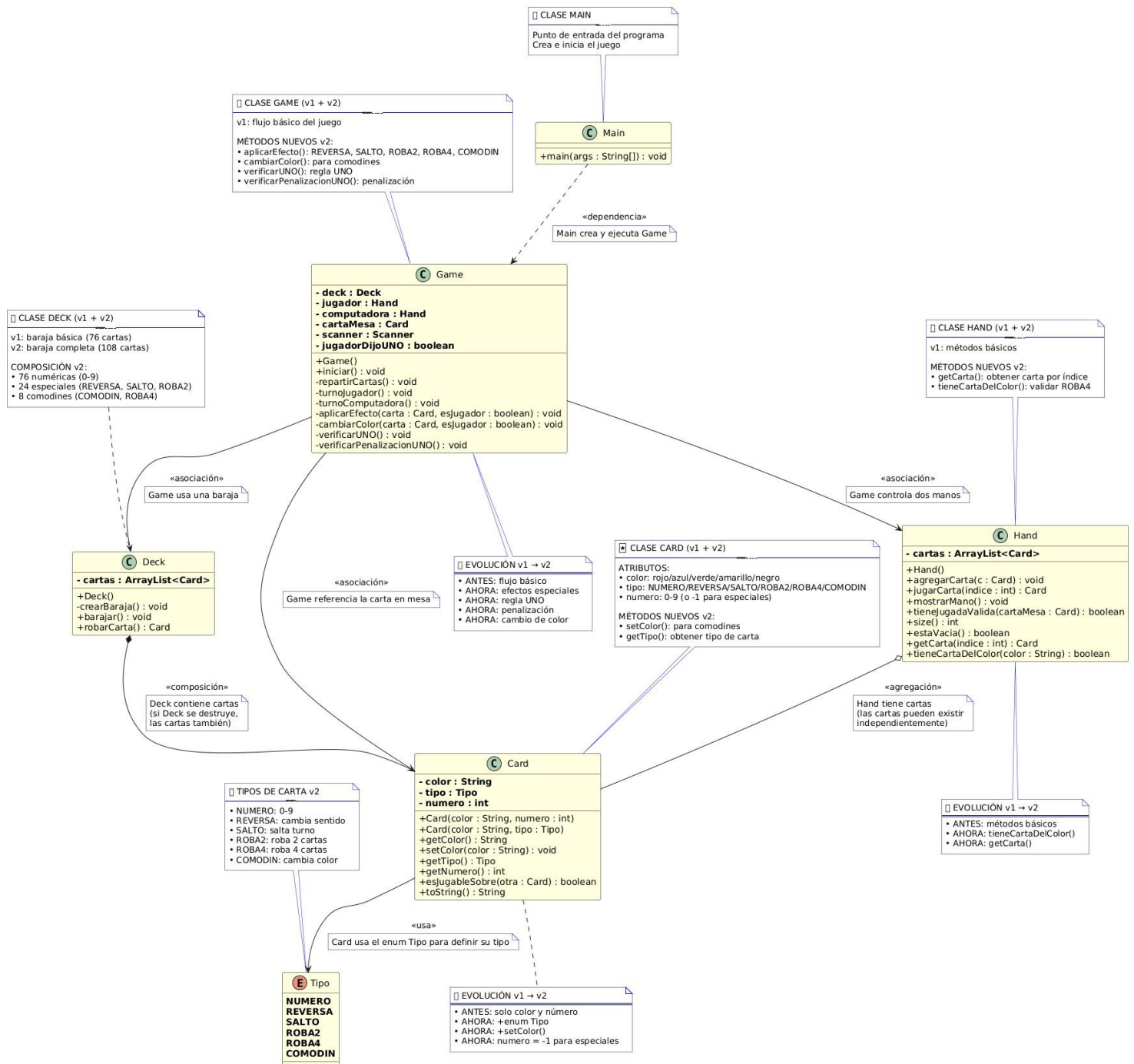


Diagrama UML

DIAGRAMA UML DE CLASES - JUEGO DE CARTAS v2

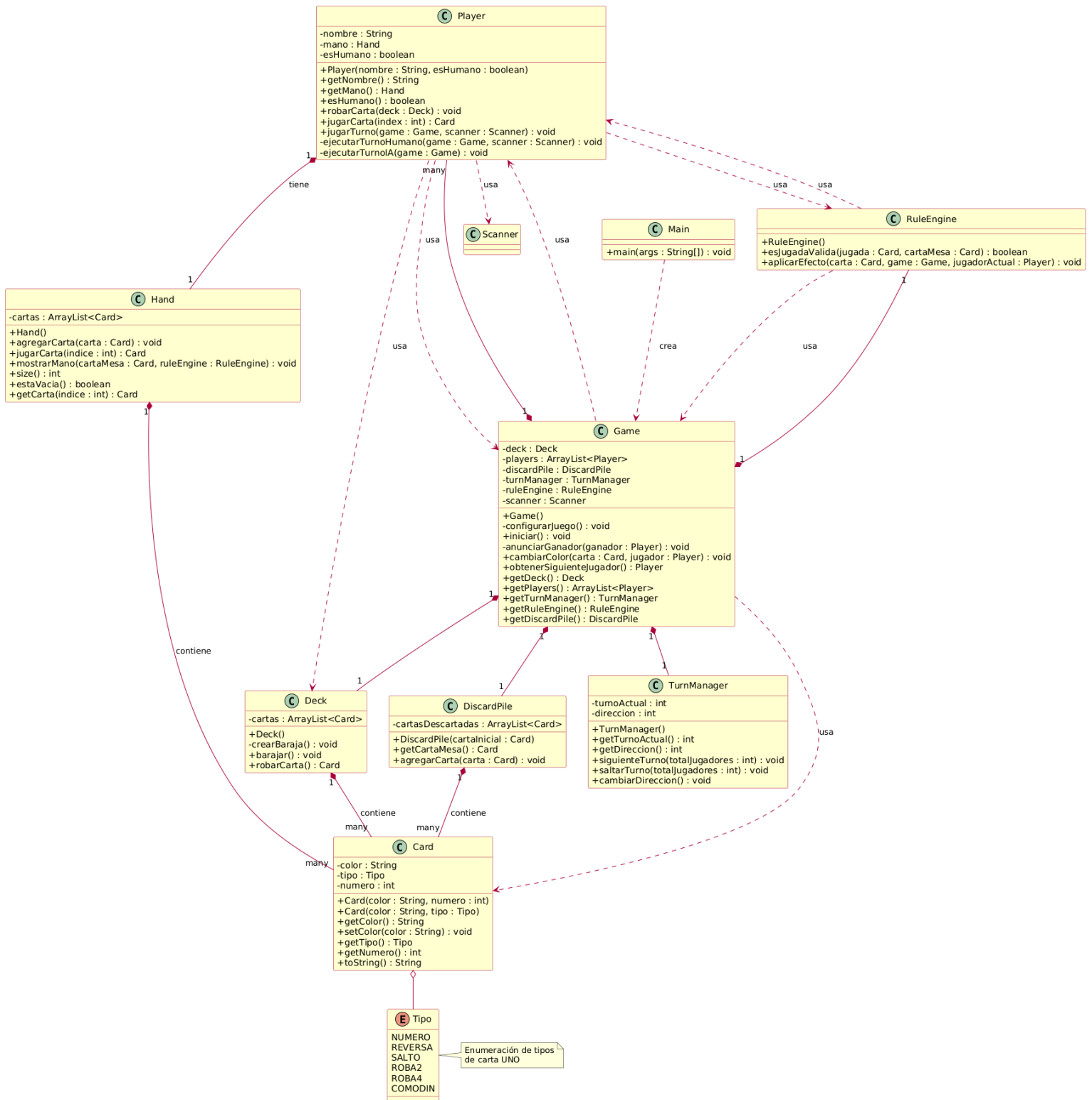


RELACIONES UML

Relación	Tipo UML
Game → Deck	Asociación
Game → Hand	Asociación
Game → Card	Asociación
Deck → Card	Composición
Hand → Card	Composición
Main → Game	Dependencia
Game → Player	Asociación
Player → Hand	Composicion

Diagrama UML v 4.0

Diagrama de Clases - CardUno (JUEGO_CARTAS_VERS4)



Código

Versión 4

CARD.JAVA

```
package JUEGO_CARTAS_VERS4;

/**
 * Representa una carta del juego UNO.
 * Una carta puede ser numerica o especial.
 */
public class Card {

    /**
     * Tipos de cartas disponibles en el juego UNO.
     */
    public enum Tipo {
        /** Carta numerica */
        NUMERO,
        /** Cambia la direccion del juego */
        REVERSA,
        /** Salta turno */
        SALTO,
        /** Obliga a robar 2 cartas */
        ROBA2,
        /** Obliga a robar 4 cartas */
        ROBA4,
        /** Permite cambiar color */
        COMODIN
    }

    private String color;
    private Tipo tipo;
    private int numero;

    /**
     * Constructor para carta numerica.
     * @param color color de la carta
     * @param numero valor numerico
     */
    public Card(String color, int numero) {
        this.color = color;
        this.numero = numero;
        this.tipo = Tipo.NUMERO;
    }

    /**
     * Constructor para carta especial.
     * @param color color de la carta
     * @param tipo tipo de carta
     */
    public Card(String color, Tipo tipo) {
        this.color = color;
        this.tipo = tipo;
        this.numero = -1;
    }

    /**
```

```

    * Obtiene el color de la carta.
    * @return color
    */
    public String getColor() {
        return color;
    }

    /**
     * Cambia el color de la carta.
     * @param color nuevo color
     */
    public void setColor(String color) {
        this.color = color;
    }

    /**
     * Obtiene el tipo de carta.
     * @return tipo
     */
    public Tipo getTipo() {
        return tipo;
    }

    /**
     * Obtiene el numero de la carta.
     * @return numero
     */
    public int getNumero() {
        return numero;
    }

    /**
     * Verifica si la carta es jugable sobre otra.
     * @param otra carta en mesa
     * @return true si es valida
     */
    public boolean esJugableSobre(Card otra) {
        return this.color.equals(otra.color) ||
            this.tipo == otra.tipo ||
            this.numero == otra.numero ||
            this.tipo == Tipo.COMODIN ||
            this.tipo == Tipo.ROBA4;
    }

    /**
     * Representacion en texto de la carta.
     * @return texto
     */
    @Override
    public String toString() {
        return tipo == Tipo.NUMERO ? color + " " + numero : color + " " + tipo;
    }
}

```

Deck.Java

```
package JUEGO_CARTAS_VERS4;

import java.util.ArrayList;
import java.util.Collections;
/**
 * Representa el mazo de cartas del juego.
 *
 * Responsable de:
 * - Crear la baraja completa
 * - Barajar cartas
 * - Permitir robar cartas
 */
public class Deck {
    /**
     * Lista de cartas del mazo.
     */
    private ArrayList<Card> cartas;
    /**
     * Construye el mazo, crea la baraja y la baraja.
     */
    public Deck() {
        cartas = new ArrayList<>();
        crearBaraja();
        barajar();
    }

    private void crearBaraja() {
        String[] colores = {"rojo", "azul", "verde", "amarillo"};
        for (String color : colores) {
            cartas.add(new Card(color, 0));
            for (int i = 1; i <= 9; i++) {
                cartas.add(new Card(color, i));
                cartas.add(new Card(color, i));
            }
            for (int j = 0; j < 2; j++) {
                cartas.add(new Card(color, Card.Tipo.SALTO));
                cartas.add(new Card(color, Card.Tipo.REVERSA));
                cartas.add(new Card(color, Card.Tipo.ROBA2));
            }
        }
        for (int i = 0; i < 4; i++) {
            cartas.add(new Card("negro", Card.Tipo.COMODIN));
            cartas.add(new Card("negro", Card.Tipo.ROBA4));
        }
    }

    /**
     * Mezcla aleatoriamente las cartas contenidas en el mazo.
     */
    public void barajar() {
        Collections.shuffle(cartas);
    }

    /**

```

```

    * Permite robar una carta del mazo.
    *
    * @return carta robada
    * @throws IllegalStateException si el mazo está vacío
    */
    public Card robarCarta() {
        if (cartas.isEmpty()) {
            throw new IllegalStateException("El mazo está vacío");
        }
        return cartas.remove(0);
    }
}

```

DiscardPile.Java

```

package JUEGO_CARTAS_VERS4;

import java.util.ArrayList;

/**
 * Representa la pila de cartas descartadas o "mesa".
 * Almacena las cartas que los jugadores van lanzando durante la partida.
 */
public class DiscardPile {

    private ArrayList<Card> cartasDescartadas;

    /**
     * Inicializa la pila de descarte con una carta inicial.
     * @param cartaInicial La primera carta que se coloca en la mesa al empezar.
     */
    public DiscardPile(Card cartaInicial) {
        cartasDescartadas = new ArrayList<>();
        cartasDescartadas.add(cartaInicial);
    }

    /**
     * Obtiene la carta que se encuentra actualmente en la parte superior de la mesa.
     * @return La última carta agregada a la pila de descarte.
     */
    public Card getCartaMesa() {
        return cartasDescartadas.get(cartasDescartadas.size() - 1);
    }

    /**
     * Agrega una nueva carta a la pila de descarte.
     * @param carta La carta jugada por un jugador.
     */
    public void agregarCarta(Card carta) {
        cartasDescartadas.add(carta);
    }
}

```

Game.Java

```
package JUEGO_CARTAS_VERS4;

import java.util.ArrayList;
import java.util.Scanner;

/**
 * Clase principal del juego UNO.
 * Se encarga de controlar el flujo del juego, los jugadores y las reglas.
 */
public class Game {

    private Deck deck;
    private ArrayList<Player> players;
    private DiscardPile discardPile;
    private TurnManager turnManager;
    private RuleEngine ruleEngine;
    private Scanner scanner;

    /**
     * Constructor del juego.
     * Inicializa todos los componentes y configura la partida.
     */
    public Game() {
        deck = new Deck();
        players = new ArrayList<>();
        scanner = new Scanner(System.in);
        turnManager = new TurnManager();
        ruleEngine = new RuleEngine();
        configurarJuego();
    }

    private void configurarJuego() {
        System.out.print("Ingresa tu nombre: ");
        String nombre = scanner.nextLine();
        players.add(new Player(nombre, true));
        players.add(new Player("Pepe", false));
        players.add(new Player("Tona", false));
        players.add(new Player("Mari", false));

        for (Player p : players) {
            for (int i = 0; i < 7; i++) {
                p.robearCarta(deck);
            }
        }

        Card primera;
        do {
            primera = deck.robearCarta();
        } while (primera.getTipo() != Card.Tipo.NUMERO);

        discardPile = new DiscardPile(primera);
    }

    /**
```

```

* Inicia el juego y controla los turnos hasta que alguien gane.
*/
public void iniciar() {
    System.out.println("Bienvenido al UNO Version Software Engineering!");
    while (true) {
        Player jugadorActual = players.get(turnManager.getTurnoActual());
        System.out.println("\n=====");
        System.out.println("TURNO DE: " + jugadorActual.getNombre());
        System.out.println("CARTA EN MESA: " + discardPile.getCartaMesa());
        System.out.println("=====");
        jugadorActual.jugarTurno(this, scanner);
        if (jugadorActual.getMano().estaVacía()) {
            anunciarGanador(jugadorActual);
            break;
        }
        turnManager.siguienteTurno(players.size());
    }
}

private void anunciarGanador(Player ganador) {
    System.out.println("\n" + ganador.getNombre() + " HA GANADO EL JUEGO!");
}

/**
 * Permite cambiar el color de una carta comodín.
 * @param carta La carta a la que se le cambiará el color.
 * @param jugador El jugador que está realizando el cambio.
 */
public void cambiarColor(Card carta, Player jugador) {
    if (jugador.esHumano()) {
        String color = "";
        while (true) {
            System.out.print("Elige nuevo color (rojo, azul, verde, amarillo): ");
            color = scanner.next().toLowerCase();
            if (color.equals("rojo") || color.equals("azul") ||
                color.equals("verde") || color.equals("amarillo")) {
                break;
            }
            System.out.println("Color no valido.");
        }
        carta.setColor(color);
    } else {
        String[] colores = {"rojo", "azul", "verde", "amarillo"};
        String elegido = colores[(int) (Math.random() * 4)];
        carta.setColor(elegido);
        System.out.println("La IA cambio el color a: " + elegido);
    }
}

/**
 * Obtiene el siguiente jugador segun el turno actual y direccion.
 * @return La instancia del objeto Player que sigue en el orden de juego.
 */
public Player obtenerSiguienteJugador() {
    int siguienteIndice = (turnManager.getTurnoActual()
        + turnManager.getDireccion())

```



```

        + players.size()) % players.size();
    return players.get(siguiienteIndice);
}

/**
 * Obtiene el mazo principal del juego.
 * @return El objeto Deck actual.
 */
public Deck getDeck() {
    return deck;
}

/**
 * Obtiene la lista de jugadores participantes.
 * @return ArrayList con los jugadores.
 */
public ArrayList<Player> getPlayers() {
    return players;
}

/**
 * Obtiene el gestor de turnos del juego.
 * @return El objeto TurnManager actual.
 */
public TurnManager getTurnManager() {
    return turnManager;
}

/**
 * Obtiene el motor de reglas aplicado al juego.
 * @return El objeto RuleEngine actual.
 */
public RuleEngine getRuleEngine() {
    return ruleEngine;
}

/**
 * Obtiene la pila de descarte actual.
 * @return El objeto DiscardPile con las cartas en mesa.
 */
public DiscardPile getDiscardPile() {
    return discardPile;
}
}

```

Hand.Java

```

package JUEGO_CARTAS_VERS4;

import java.util.ArrayList;
/**
 * Representa la mano de un jugador en el juego UNO
 *
 * Encapsula:
 * - La colección de cartas del jugador

```

```

* - Operaciones como agregar, jugar y consultar cartas
*/
public class Hand {
    /**
     * Lista de cartas en la mano
     */
    private ArrayList<Card> cartas;

    /**
     * Crea una nueva mano vacía para el jugador.
     */
    public Hand() {
        cartas = new ArrayList<>();
    }

    /**
     * Añade una carta a la colección del jugador.
     * @param carta La carta que se desea agregar.
     */
    public void agregarCarta(Card carta) {
        cartas.add(carta);
    }

    /**
     * Remueve y retorna una carta de la mano según su índice
     *
     * @param indice posición de la carta en la mano
     * @return carta jugada
     * @throws IllegalArgumentException si el índice es inválido
     */
    public Card jugarCarta(int indice) {
        if (indice >= 0 && indice < cartas.size()) {
            return cartas.remove(indice);
        }
        throw new IllegalArgumentException("Índice inválido");
    }

    /**
     * Muestra las cartas del jugador indicando cuáles son jugables
     *
     * @param cartaMesa carta actual en mesa
     * @param ruleEngine motor de reglas
     */
    public void mostrarMano(Card cartaMesa, RuleEngine ruleEngine) {
        System.out.println("\n(─█─) TUS CARTAS:\n");
        for (int i = 0; i < cartas.size(); i++) {
            Card c = cartas.get(i);
            if (ruleEngine.esJugadaValida(c, cartaMesa)) {
                System.out.println("--> [" + i + "] " + c + " <<< JUGABLE");
            } else {
                System.out.println("    [" + i + "] " + c);
            }
        }
    }
}

/**

```

```

    * Obtiene el numero de cartas en la mano
    *
    * @return cantidad de cartas
    */
    public int size() {
        return cartas.size();
    }

    /**
     * Indica si la mano está vacia
     *
     * @return true si no hay cartas
     */
    public boolean estaVacia() {
        return cartas.isEmpty();
    }

    /**
     * Obtiene una carta sin removerla de la mano
     *
     * @param indice posición de la carta
     * @return carta en la posición indicada
     */
    public Card getCarta(int indice) {
        return cartas.get(indice);
    }
}

```

Main.Java

```

package JUEGO_CARTAS_VERS4;

/**
 * Clase de entrada para la aplicación.
 * Contiene el método main que lanza la ejecución del juego UNO.
 */
public class Main {

    /**
     * Constructor por defecto de la clase Main.
     */
    public Main() {}

    /**
     * Punto de entrada del programa. Crea e inicia una nueva partida.
     * @param args Argumentos de línea de comandos (no utilizados).
     */
    public static void main(String[] args) {
        Game juego = new Game();
        juego.iniciar();
    }
}

```

Player.Java

```
package JUEGO_CARTAS_VERS4;

import java.util.Scanner;

/**
 * Representa a un jugador del juego UNO.
 * Puede ser humano o IA.
 *
 * Se encarga de:
 * - Guardar su mano de cartas
 * - Robar cartas
 * - Jugar cartas
 * - Ejecutar su turno
 */
public class Player {

    private String nombre;
    private Hand mano;
    private boolean esHumano;

    /**
     * Crea un nuevo jugador con un nombre y define si es humano o bot.
     * @param nombre El nombre identificador del jugador.
     * @param esHumano true si el jugador es controlado por una persona, false si es IA.
     */
    public Player(String nombre, boolean esHumano) {
        this.nombre = nombre;
        this.esHumano = esHumano;
        this.mano = new Hand();
    }

    /**
     * Obtiene el nombre del jugador.
     * @return nombre del jugador.
     */
    public String getNombre() { return nombre; }

    /**
     * Obtiene la mano de cartas actual del jugador.
     * @return El objeto Hand asociado al jugador.
     */
    public Hand getMano() { return mano; }

    /**
     * Verifica si el jugador es humano.
     * @return true si es humano.
     */
    public boolean esHumano() { return esHumano; }

    /**
     * Permite al jugador robar una carta del mazo y agregarla a su mano.
     * @param deck El mazo del que se extraerá la carta.
     */
    public void robarCarta(Deck deck) {
```

```

        mano.agregarCarta(deck.robCarta());
    }

    /**
     * Permite jugar una carta de la mano según su índice de posición.
     * @param index El índice de la carta en la mano.
     * @return La carta seleccionada para jugar.
     */
    public Card jugarCarta(int index) {
        return mano.jugarCarta(index);
    }

    /**
     * Ejecuta el turno del jugador.
     * @param game Instancia del juego para acceder a la lógica y componentes.
     * @param scanner Objeto Scanner para recibir la entrada del usuario si es humano.
     */
    public void jugarTurno(Game game, Scanner scanner) {
        if (esHumano) {
            ejecutarTurnoHumano(game, scanner);
        } else {
            ejecutarTurnoIA(game);
        }
    }

    private void ejecutarTurnoHumano(Game game, Scanner scanner) {
        RuleEngine ruleEngine = game.getRuleEngine();
        DiscardPile discardPile = game.getDiscardPile();
        Deck deck = game.getDeck();

        while (true) {
            mano.mostrarMano(discardPile.getCartaMesa(), ruleEngine);
            System.out.println("\n- Escribe numero para jugar");
            System.out.println("- Escribe -1 para robar carta: ");

            if (!scanner.hasNextInt()) {
                System.out.println("Ingresa un numero valido.");
                scanner.next();
                continue;
            }

            int opcion = scanner.nextInt();

            if (opcion == -1) {
                Card robada = deck.robCarta();
                System.out.println("Has robado: " + robada);
                if (ruleEngine.esJugadaValida(robada, discardPile.getCartaMesa())) {
                    System.out.print("Quieres jugar la carta robada? (s/n): ");
                    String r = scanner.next();
                    if (r.equalsIgnoreCase("s")) {
                        discardPile.agregarCarta(robada);
                        ruleEngine.aplicarEfecto(robada, game, this);
                        return;
                    }
                }
            }
            mano.agregarCarta(robada);
        }
    }

```

```

        return;
    }

    if (opcion < 0 || opcion >= mano.size()) {
        System.out.println("Numero fuera de rango.");
        continue;
    }

    Card cartaParaVer = mano.getCarta(opcion);
    if (!ruleEngine.esJugadaValida(cartaParaVer, discardPile.getCartaMesa())) {
        System.out.println("Esa carta no se puede jugar.");
        continue;
    }

    Card cartaJugada = mano.jugarCarta(opcion);
    System.out.println(nombre + " juega: " + cartaJugada);
    discardPile.agregarCarta(cartaJugada);
    ruleEngine.aplicarEfecto(cartaJugada, game, this);
    return;
}

}

private void ejecutarTurnoIA(Game game) {
    RuleEngine ruleEngine = game.getRuleEngine();
    DiscardPile discardPile = game.getDiscardPile();
    Deck deck = game.getDeck();

    for (int i = 0; i < mano.size(); i++) {
        Card c = mano.getCarta(i);
        if (ruleEngine.esJugadaValida(c, discardPile.getCartaMesa())) {
            Card jugada = mano.jugarCarta(i);
            discardPile.agregarCarta(jugada);
            System.out.println(nombre + " juega: " + jugada);
            ruleEngine.aplicarEfecto(jugada, game, this);
            if (mano.size() == 1) {
                System.out.println(nombre + " dice UNO!");
            }
            return;
        }
    }

    Card robada = deck.robarCarta();
    System.out.println(nombre + " roba una carta...");
    if (ruleEngine.esJugadaValida(robada, discardPile.getCartaMesa())) {
        System.out.println(nombre + " juega la robada: " + robada);
        discardPile.agregarCarta(robada);
        ruleEngine.aplicarEfecto(robada, game, this);
        if (mano.size() == 1) {
            System.out.println(nombre + " dice UNO!");
        }
    } else {
        mano.agregarCarta(robada);
    }
}

}
}

```

RuleEngine.Java

```
package JUEGO_CARTAS_VERS4;

/**
 * Motor de reglas del juego UNO
 *
 * Responsable de:
 * - Validar jugadas
 * - Aplicar efectos de cartas especiales
 *
 * Separa la lógica del juego de la interfaz y del flujo principal
 */
public class RuleEngine {

    /**
     * Constructor por defecto de RuleEngine.
     * Inicializa el motor de reglas para la validación de jugadas y efectos.
     */
    public RuleEngine() {
        // Constructor vacío explícito para documentación Javadoc
    }

    /**
     * Determina si una carta puede jugarse sobre otra
     *
     * Reglas:
     * - Mismo color
     * - Mismo número
     * - Mismo tipo
     * - Comodín y ROBA4 siempre válidos
     *
     * @param jugada carta a evaluar
     * @param cartaMesa carta actual en mesa
     * @return true si la jugada es válida
     */
    public boolean esJugadaValida(Card jugada, Card cartaMesa) {

        if (jugada.getTipo() == Card.Tipo.COMODIN || jugada.getTipo() == Card.Tipo.ROBA4) {
            return true;
        }

        if (jugada.getColor().equals(cartaMesa.getColor())) {
            return true;
        }

        if (jugada.getTipo() == Card.Tipo.NUMERO && cartaMesa.getTipo() == Card.Tipo.NUMERO) {
            return jugada.getNumero() == cartaMesa.getNumero();
        }

        return jugada.getTipo() == cartaMesa.getTipo();
    }

    /**
     * Aplica el efecto de una carta especial
     *
     * Ejemplos:
     */
}
```

```

* - SALTO → omite turno
* - REVERSA → cambia dirección
* - ROBA2 / ROBA4 → roba cartas y pierde turno
*
* @param carta carta jugada
* @param game instancia del juego
* @param jugadorActual jugador que jugó la carta
*/
public void aplicarEfecto(Card carta, Game game, Player jugadorActual) {

    switch (carta.getTipo()) {

        case SALTO:
            game.getTurnManager().saltarTurno(game.getPlayers().size());
            break;

        case REVERSA:
            if (game.getPlayers().size() == 2) {
                game.getTurnManager().saltarTurno(game.getPlayers().size());
            } else {
                game.getTurnManager().cambiarDireccion();
            }
            break;

        case ROBA2:
            Player siguiente = game.obtenerSiguienteJugador();
            for (int i = 0; i < 2; i++) {
                siguiente.robarCarta(game.getDeck());
            }
            game.getTurnManager().saltarTurno(game.getPlayers().size());
            break;

        case ROBA4:
            Player siguiente4 = game.obtenerSiguienteJugador();
            for (int i = 0; i < 4; i++) {
                siguiente4.robarCarta(game.getDeck());
            }
            game.cambiarColor(carta, jugadorActual);
            game.getTurnManager().saltarTurno(game.getPlayers().size());
            break;

        case COMODIN:
            game.cambiarColor(carta, jugadorActual);
            break;

        default:
            break;
    }
}
}

```


TurnManager.Java

```
package JUEGO_CARTAS_VERS4;

/**
 * Controla el orden de los turnos del juego.
 * Maneja que jugador juega y en que direccion.
 */
public class TurnManager {

    private int turnoActual;
    private int direccion;

    /**
     * Inicializa el gestor de turnos con el primer jugador y dirección normal.
     */
    public TurnManager() {
        turnoActual = 0;
        direccion = 1;
    }

    /**
     * Obtiene el índice del jugador que tiene el turno actualmente.
     * @return Índice numérico del jugador.
     */
    public int getTurnoActual() {
        return turnoActual;
    }

    /**
     * Obtiene la dirección actual de la ronda (1 para normal, -1 para reversa).
     * @return El valor de la dirección.
     */
    public int getDireccion() {
        return direccion;
    }

    /**
     * Avanza al siguiente jugador segun la direccion.
     * @param totalJugadores Cantidad total de jugadores en la partida para calcular el ciclo.
     */
    public void siguienteTurno(int totalJugadores) {
        turnoActual = (turnoActual + direccion + totalJugadores) % totalJugadores;
    }

    /**
     * Salta el turno de un jugador.
     * Se usa en cartas como SALTO o ROBA.
     * @param totalJugadores Cantidad total de jugadores en la partida.
     */
    public void saltarTurno(int totalJugadores) {
```

```

    siguienteTurno(totalJugadores);
    siguienteTurno(totalJugadores);
}

/**
 * Invierte la dirección de la rotación de los turnos.
 */
public void cambiarDireccion() {
    direccion *= -1;
}
}

```

Entrega

Versión 1 (Semanas 1-2)

- UML inicial
- v1 funcional
- Repositorio Git

Versión 2 (Semanas 3-4)

- UML actualizado v2
- Código con cartas especiales

Versión 3 (Semanas 5-6)

- V3 refactorizada (Entrega funcional)
- Documento: ¿Qué se mejoró y por qué?
- Diagrama UML actualizado
- README del juego

Versión 3 (Semanas 6-7)

- Documentación Final: Archivo Word actualizado que incluye el historial de versiones v4.0,

requisitos no funcionales de calidad y la conclusión final del proyecto.

- Código Fuente Documentado: Carpeta src con los archivos .java que incluyen los bloques de comentarios / ... */ validados.
- Suite de Pruebas Unitarias: Archivo PruebasUno.java con los casos de prueba JUnit documentados y listos para ejecución.
- Sitio Web de Javadoc: Carpeta doc generada por el IDE (Eclipse), que contiene el archivo index.html para la navegación técnica del proyecto.
- Diagrama UML Actualizado: Representación visual de las clases finales, destacando la independencia del motor de reglas y la estructura de múltiples jugadores.

